

AAI-590 Capstone — Group 4

Module 4 | Notebook 3: MFCC Feature Extraction Pipeline

Project: ML-Based Murmur Detection for RHD Screening Using Phonocardiogram Data

Dataset: CirCor DigiScope Phonocardiogram Dataset v1.0.3 (PhysioNet)

Authors: Shiva Bathula · Kaushik M · Pavan Kumar Bellary

Notebook Goals

1. Session recovery — restore files if Colab restarted
 2. Define feature extraction function (MFCC + spectral features)
 3. Extract features for all 3,163 recordings
 4. Patient-level train/test split (no data leakage)
 5. Save feature matrix to CSV for use in Model Training notebook
 6. Feature distribution analysis and sanity checks
 7. Push feature matrix and notebook to GitHub
-

1. Mount Drive + load paths + verify

```
In [6]: from google.colab import drive
from pathlib import Path
import pandas as pd
import os

# Mount Drive
drive.mount('/content/drive')

# — All paths from Drive —
DRIVE_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4')
DATA_ROOT = DRIVE_DIR / 'data'
AUDIO_DIR = DATA_ROOT / 'training_data'
LABEL_CSV = DATA_ROOT / 'training_data.csv'
INDEX_CSV = DATA_ROOT / 'circor_file_index.csv'
OUTPUT_DIR = Path('/content/features')
OUTPUT_DIR.mkdir(exist_ok=True)

# — Verify —
print('— Verifying Drive paths —')
for name, path in [('AUDIO_DIR', AUDIO_DIR),
                   ('LABEL_CSV', LABEL_CSV),
                   ('INDEX_CSV', INDEX_CSV)]:
    exists = Path(path).exists()
    print(f'{name} {"exists" if exists else "does not exist"}')

wav_count = len(list(AUDIO_DIR.glob('*.wav')))
print(f'\nWAV files: {wav_count}')
```

```
# — Load dataframes —
file_index = pd.read_csv(INDEX_CSV)
df_labels = pd.read_csv(LABEL_CSV)
print(f'file_index : {file_index.shape}')
print(f'df_labels : {df_labels.shape}')
print('\n Ready — now run Cell 3 (feature function) and Cell 4 (extracti
print(' SKIP Cell 1 and Cell 2 entirely.')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

— Verifying Drive paths —

```

AUDIO_DIR: /content/drive/MyDrive/AAI590_Capstone_Group4/data/training_data
LABEL_CSV: /content/drive/MyDrive/AAI590_Capstone_Group4/data/training_data
INDEX_CSV: /content/drive/MyDrive/AAI590_Capstone_Group4/data/
circor_file_index.csv
```

```

WAV files: 3163
file_index : (3163, 5)
df_labels : (942, 23)
```

```

Ready — now run Cell 3 (feature function) and Cell 4 (extraction loop).
SKIP Cell 1 and Cell 2 entirely.
```

3. Feature Extraction Function

```
In [7]: # — Feature extraction configuration —
N_MFCC      = 13      # MFCC coefficients (standard for biosignal classific
N_MELS      = 64      # Mel filter banks
FMAX        = 1000    # Max frequency (Hz) — heart sounds concentrate below
SR_TARGET   = 4000    # Native sample rate — no resampling needed
MAX_DUR     = 30      # Cap at 30s to avoid memory issues on very long reco

def extract_features(wav_path: str) -> dict:
    """
    Extract acoustic features from a single PCG WAV recording.

    Feature set:
    - 13 MFCC mean values      (spectral envelope shape)
    - 13 MFCC std values      (temporal variability)
    - Spectral centroid mean   (brightness / frequency centre)
    - Spectral bandwidth mean  (frequency spread)
    - Spectral rolloff mean    (energy concentration)
    - Spectral flatness mean   (noise vs tonal character)
    - Zero crossing rate mean  (signal roughness)
    - RMS energy mean          (overall amplitude)
    - RMS energy std           (amplitude variability)
    Total: 35 features per recording

    Returns dict of feature_name: value, or None if extraction fails.
    """
    try:
        y, sr = librosa.load(wav_path, sr=None, duration=MAX_DUR)

        # — MFCC (13 coefficients, mean + std over time) —
        mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=N_MFCC)
        mfcc_mean = mfcc.mean(axis=1) # shape: (13,)
```

```

mfcc_std = mfcc.std(axis=1) # shape: (13,)

# — Spectral features —
centroid = librosa.feature.spectral_centroid(y=y, sr=sr).mean()
bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=sr).mean()
rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr, roll_per
flatness = librosa.feature.spectral_flatness(y=y).mean()

# — Temporal features —
zcr = librosa.feature.zero_crossing_rate(y).mean()
rms = librosa.feature.rms(y=y)
rms_mean = rms.mean()
rms_std = rms.std()

# — Build feature dict —
features = {}
for i, val in enumerate(mfcc_mean):
    features[f'mfcc_mean_{i+1}'] = val
for i, val in enumerate(mfcc_std):
    features[f'mfcc_std_{i+1}'] = val
features['spectral_centroid'] = centroid
features['spectral_bandwidth'] = bandwidth
features['spectral_rolloff'] = rolloff
features['spectral_flatness'] = flatness
features['zcr'] = zcr
features['rms_mean'] = rms_mean
features['rms_std'] = rms_std

return features

except Exception as e:
    print(f'Error processing {wav_path}: {e}')
    return None

# — Test on one file —
test_wav = list(AUDIO_DIR.glob('*.wav'))[0]
test_features = extract_features(str(test_wav))
print(f'Test file      : {test_wav.name}')
print(f'Feature count  : {len(test_features)}')
print(f'Feature names   : {list(test_features.keys())}')

```

Test file : 50720_AV.wav

Feature count : 33

Feature names : ['mfcc_mean_1', 'mfcc_mean_2', 'mfcc_mean_3', 'mfcc_mean_4', 'mfcc_mean_5', 'mfcc_mean_6', 'mfcc_mean_7', 'mfcc_mean_8', 'mfcc_mean_9', 'mfcc_mean_10', 'mfcc_mean_11', 'mfcc_mean_12', 'mfcc_mean_13', 'mfcc_std_1', 'mfcc_std_2', 'mfcc_std_3', 'mfcc_std_4', 'mfcc_std_5', 'mfcc_std_6', 'mfcc_std_7', 'mfcc_std_8', 'mfcc_std_9', 'mfcc_std_10', 'mfcc_std_11', 'mfcc_std_12', 'mfcc_std_13', 'spectral_centroid', 'spectral_bandwidth', 'spectral_rolloff', 'spectral_flatness', 'zcr', 'rms_mean', 'rms_std']

4. Extract Features for All 3,163 Recordings

```

In [8]: # — Full feature extraction loop —
# Expected runtime: ~15-25 minutes for 3,163 recordings in Colab

records = []

```

```

failed      = []
wav_files   = sorted(AUDIO_DIR.glob('*.wav'))

print(f'Extracting features from {len(wav_files)} recordings...')
print('This will take approximately 15-25 minutes.')
print()

for wav in tqdm(wav_files, desc='Feature extraction'):
    parts = wav.stem.split('_')
    pid   = int(parts[0]) if parts[0].isdigit() else None
    site  = parts[1] if len(parts) > 1 else 'unknown'

    feats = extract_features(str(wav))

    if feats is not None:
        feats['patient_id'] = pid
        feats['site']       = site
        feats['wav_file']   = wav.name
        records.append(feats)
    else:
        failed.append(wav.name)

feature_df = pd.DataFrame(records)

print(f'\nExtraction complete.')
print(f'Successful : {len(records)}')
print(f'Failed      : {len(failed)}')
print(f'Feature matrix shape: {feature_df.shape}')
if failed:
    print(f'Failed files: {failed[:5]}')

```

Extracting features from 3163 recordings...
This will take approximately 15-25 minutes.

```

Feature extraction: 100%|██████████| 3163/3163 [05:20<00:00, 9.87it/s]
Extraction complete.
Successful : 3163
Failed      : 0
Feature matrix shape: (3163, 36)

```

5. Merge Labels & Patient-Level Train/Test Split

```

In [9]: # — Merge outcome labels —————
label_cols = ['Patient ID', 'Outcome', 'Murmur']
feature_df = feature_df.merge(
    df_labels[label_cols],
    left_on='patient_id', right_on='Patient ID',
    how='left'
).drop(columns=['Patient ID'])

# Binary encode Outcome for modelling
feature_df['outcome_binary'] = (feature_df['Outcome'] == 'Abnormal').astype(int)

print(f'Feature matrix with labels: {feature_df.shape}')
print(f'\nOutcome distribution in feature matrix:')
print(feature_df['Outcome'].value_counts())
print(f'\nMissing values: {feature_df.isnull().sum().sum()}')

```

```

# — Patient-level train/test split (80/20) —————
# CRITICAL: split on patient_id, not recording
# This prevents data leakage from multi-recording patients
import numpy as np
np.random.seed(42)

unique_patients = feature_df['patient_id'].dropna().unique()
np.random.shuffle(unique_patients)

split_idx = int(len(unique_patients) * 0.80)
train_pids = set(unique_patients[:split_idx])
test_pids = set(unique_patients[split_idx:])

feature_df['split'] = feature_df['patient_id'].apply(
    lambda x: 'train' if x in train_pids else 'test'
)

train_df = feature_df[feature_df['split'] == 'train']
test_df = feature_df[feature_df['split'] == 'test']

print(f'\n— Patient-level 80/20 split —————')
print(f'Total patients : {len(unique_patients)}')
print(f'Train patients : {len(train_pids)} → {len(train_df)} recordings')
print(f'Test patients : {len(test_pids)} → {len(test_df)} recordings')
print(f'\nTrain outcome distribution:')
print(train_df['Outcome'].value_counts())
print(f'\nTest outcome distribution:')
print(test_df['Outcome'].value_counts())

```

Feature matrix with labels: (3163, 39)

Outcome distribution in feature matrix:

```

Outcome
Normal      1632
Abnormal    1531
Name: count, dtype: int64

```

Missing values: 0

```

— Patient-level 80/20 split —————
Total patients : 942
Train patients : 753 → 2523 recordings
Test patients  : 189 → 640 recordings

```

Train outcome distribution:

```

Outcome
Normal      1308
Abnormal    1215
Name: count, dtype: int64

```

Test outcome distribution:

```

Outcome
Normal      324
Abnormal    316
Name: count, dtype: int64

```

```
# — Save full feature matrix and splits —
FEATURE_ALL = OUTPUT_DIR / 'features_all.csv'
FEATURE_TRAIN = OUTPUT_DIR / 'features_train.csv'
FEATURE_TEST = OUTPUT_DIR / 'features_test.csv'

feature_df.to_csv(FEATURE_ALL, index=False)
train_df.to_csv(FEATURE_TRAIN, index=False)
test_df.to_csv(FEATURE_TEST, index=False)

print('Files saved:')
for f in [FEATURE_ALL, FEATURE_TRAIN, FEATURE_TEST]:
    size = Path(f).stat().st_size / 1024
    print(f' {f.name} ({size:.1f} KB)')
```

```
files saved:
features_all.csv  (1268.2 KB)
features_train.csv (1012.2 KB)
features_test.csv  (256.4 KB)
```

```
# — MFCC feature distributions by Outcome —
mfcc_cols = [f'mfcc_mean_{i}' for i in range(1, N_MFCC+1)]

fig, axes = plt.subplots(3, 5, figsize=(18, 10))
axes = axes.flatten()

for i, col in enumerate(mfcc_cols):
    if col in feature_df.columns:
        for outcome, color in [('Normal', GREEN), ('Abnormal', RED)]:
            vals = feature_df[feature_df['Outcome'] == outcome][col]
            axes[i].hist(vals, bins=30, alpha=0.6, color=color,
                        label=outcome, density=True)
            axes[i].set_title(f'MFCC {i+1}', fontweight='bold', fontsize=10)
            axes[i].set_xlabel('Value')
            if i == 0: axes[i].legend(fontsize=8)

# Hide unused subplots
for j in range(len(mfcc_cols), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('MFCC Feature Distributions – Normal vs Abnormal\n'
            '(density normalised)', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'mfcc_distributions.png', bbox_inches='tight')
plt.show()

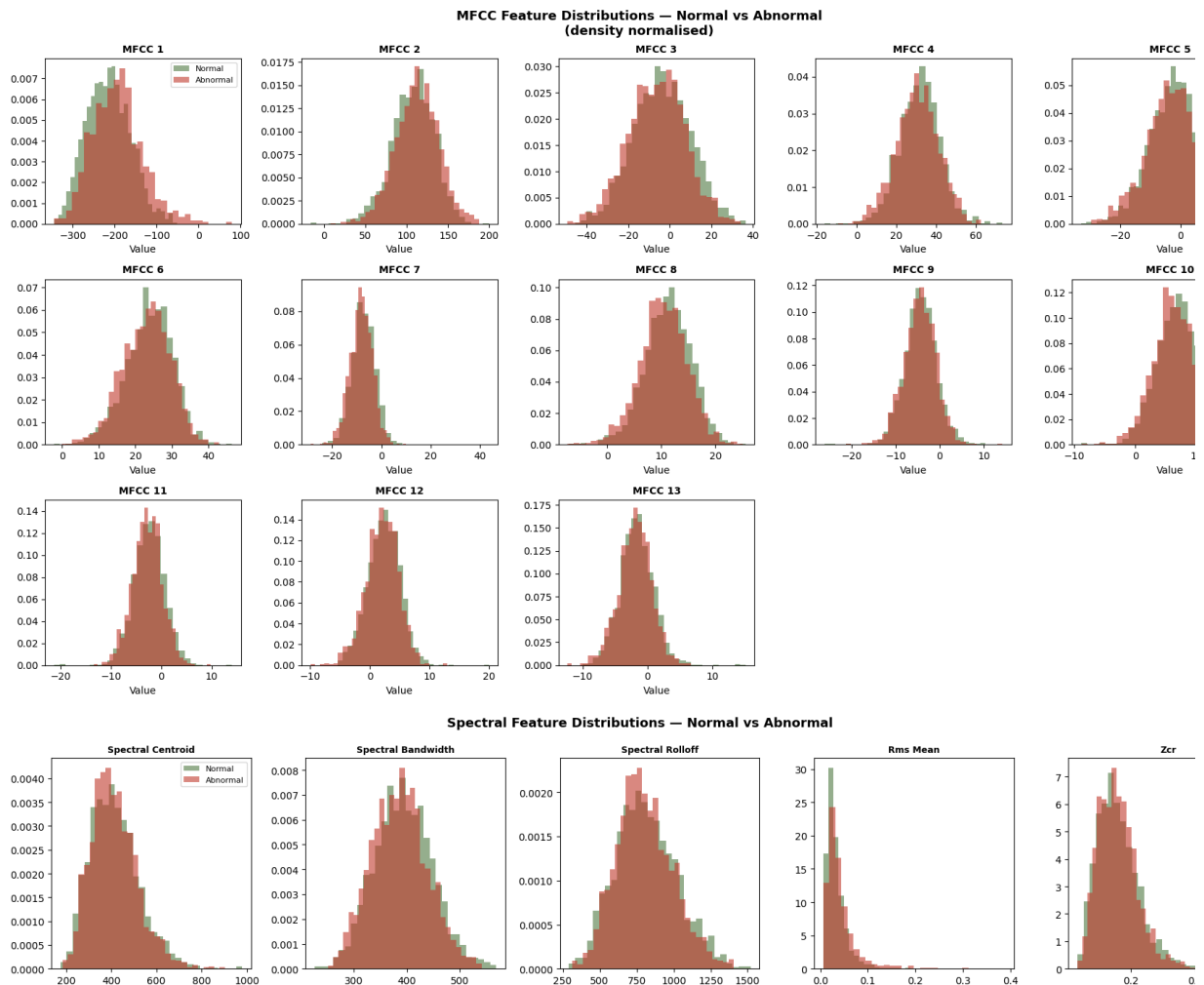
# — Spectral feature comparison —
spectral_cols = ['spectral_centroid', 'spectral_bandwidth',
                'spectral_rolloff', 'rms_mean', 'zcr']
```

```

fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(spectral_cols):
    if col in feature_df.columns:
        for outcome, color in [('Normal', GREEN), ('Abnormal', RED)]:
            vals = feature_df[feature_df['Outcome'] == outcome][col]
            axes[i].hist(vals, bins=30, alpha=0.6, color=color,
                        label=outcome, density=True)
            axes[i].set_title(col.replace('_', ' ').title(),
                            fontweight='bold', fontsize=9)
        if i == 0: axes[i].legend(fontsize=8)

plt.suptitle('Spectral Feature Distributions — Normal vs Abnormal',
            fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'spectral_distributions.png', bbox_inches='tight')
plt.show()

```



```

In [12]: # — MFCC correlation heatmap —
mfcc_mean_cols = [f'mfcc_mean_{i}' for i in range(1, N_MFCC+1)]
available_cols = [c for c in mfcc_mean_cols if c in feature_df.columns]

fig, ax = plt.subplots(figsize=(10, 8))
corr = feature_df[available_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, ax=ax,
            xticklabels=[f'MFCC{i}' for i in range(1, len(available_cols))],
            yticklabels=[f'MFCC{i}' for i in range(1, len(available_cols))],
            ax.set_title('MFCC Inter-Coefficient Correlation Matrix\n')

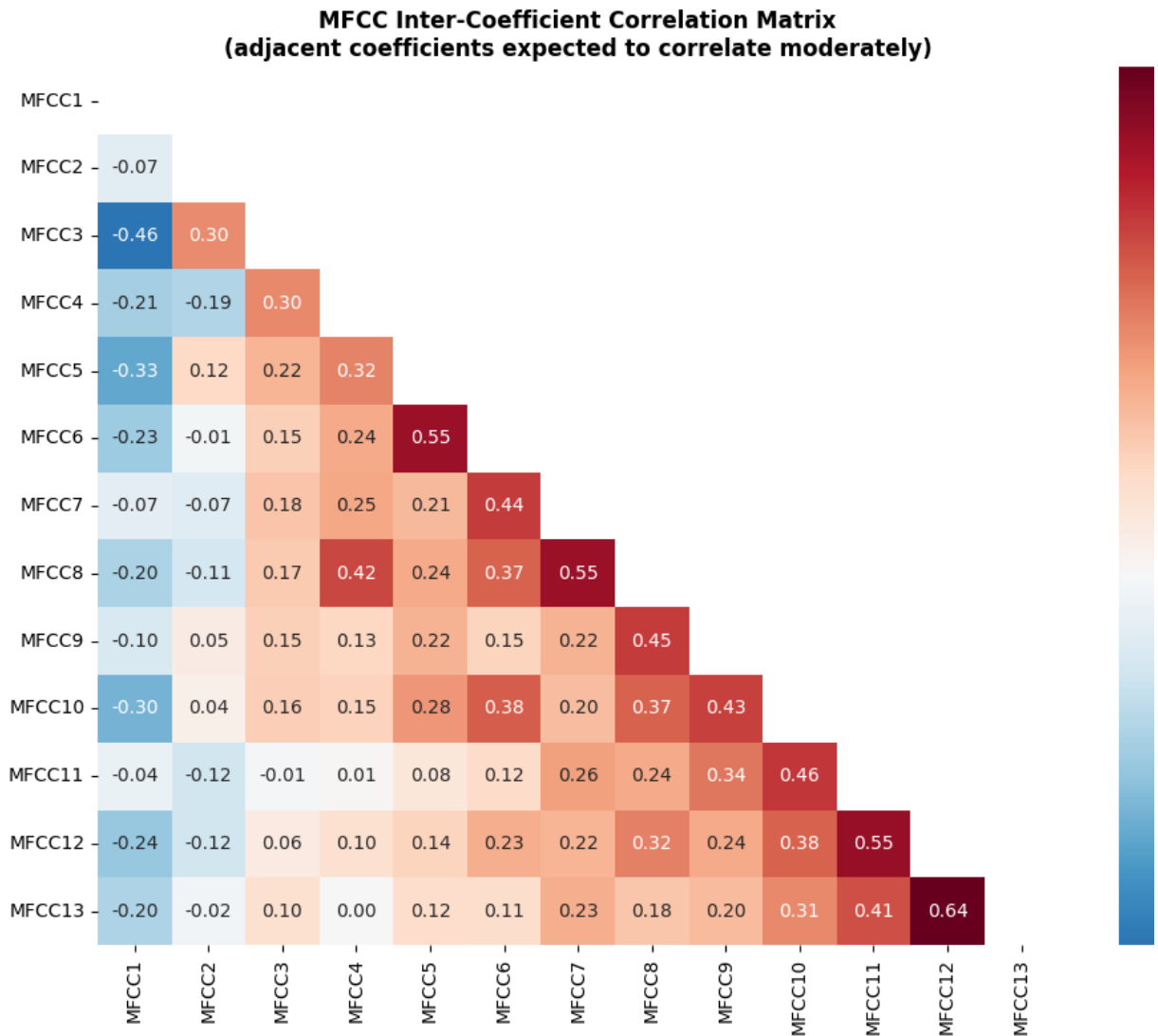
```

```

        '(adjacent coefficients expected to correlate moderately)',
        fontweight='bold')
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'mfcc_correlation.png', bbox_inches='tight')
plt.show()

print('— Feature extraction summary —')
print(f'Total features per recording : {len([c for c in feature_df.columns])}')
print(f'Training set : {len(train_df)} recordings')
print(f'Test set : {len(test_df)} recordings')
print(f'Feature files saved to : {OUTPUT_DIR}')

```



```

— Feature extraction summary —
Total features per recording : 33
Training set : 2523 recordings
Test set : 640 recordings
Feature files saved to : /content/features

```

8. Push to GitHub

```

In [13]: # — Push feature matrix, plots, and notebook to GitHub —
import subprocess, shutil, os, getpass

os.chdir('/content')

```



```

GITHUB_USER = input('GitHub username: ')
GITHUB_TOKEN = getpass.getpass('GitHub PAT: ')
REPO_NAME = 'AAI590-Capstone-Group4-RHD-Murmur-Detection'
REMOTE_URL = f'https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GITHUB_

def run(cmd, cwd='/content/repo'):
    r = subprocess.run(cmd, shell=True, cwd=cwd, capture_output=True, tex
    out = (r.stdout + r.stderr).strip()
    if out: print(out)
    return r.returncode

# Fresh clone
run('rm -rf /content/repo', cwd='/content')
run(f'git clone {REMOTE_URL} /content/repo', cwd='/content')

# Create folders
os.makedirs('/content/repo/notebooks', exist_ok=True)
os.makedirs('/content/repo/data/features', exist_ok=True)
os.makedirs('/content/repo/outputs/m4_features', exist_ok=True)

# Copy feature CSVs
for csv_file in ['features_all.csv', 'features_train.csv', 'features_test
    src = OUTPUT_DIR / csv_file
    if src.exists():
        shutil.copy2(str(src), f'/content/repo/data/features/{csv_file}')
        size = src.stat().st_size / 1024
        print(f'    {csv_file} ({size:.1f} KB)')

# Copy output figures
for fig_file in OUTPUT_DIR.glob('*.png'):
    shutil.copy2(str(fig_file),
        f'/content/repo/outputs/m4_features/{fig_file.name}')
    print(f'    {fig_file.name}')

# Copy this notebook
for f in os.listdir('/content'):
    if ('M3_02' in f or 'EDA' in f or 'M4' in f or 'Feature' in f) and f.
        shutil.copy2(f'/content/{f}',
            '/content/repo/notebooks/AAI590_M4_03_FeatureExtract
        print(f'    Notebook: {f}')
        break

# Git push
run('git config user.email "shiva.bathula@sandiego.edu"')
run('git config user.name "Shiva Naga Vara Prasad Bathula"')
run('git add -A')
run('git status --short')
run('git commit -m "M4: Add feature extraction notebook, feature matrix C
rc = run('git push origin main')
print(f'\nPush: {" " if rc==0 else ""} code={rc}')
print(f'    https://github.com/{GITHUB_USER}/{REPO_NAME}')

```

```
GitHub username: ShivaNagaVaraPrasadBathula
GitHub PAT: .....
Cloning into '/content/repo'...
└─ features_all.csv (1268.2 KB)
└─ features_train.csv (1012.2 KB)
└─ features_test.csv (256.4 KB)
└─ mfcc_correlation.png
└─ spectral_distributions.png
└─ mfcc_distributions.png
A data/features/features_all.csv
A data/features/features_test.csv
A data/features/features_train.csv
A outputs/m4_features/mfcc_correlation.png
A outputs/m4_features/mfcc_distributions.png
A outputs/m4_features/spectral_distributions.png
[main d887250] M4: Add feature extraction notebook, feature matrix CSVs, and 6 files changed, 6329 insertions(+)
 create mode 100644 data/features/features_all.csv
 create mode 100644 data/features/features_test.csv
 create mode 100644 data/features/features_train.csv
 create mode 100644 outputs/m4_features/mfcc_correlation.png
 create mode 100644 outputs/m4_features/mfcc_distributions.png
 create mode 100644 outputs/m4_features/spectral_distributions.png
To https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Murder-Detection.git
 1c97d78..d887250 main -> main

Push: └─ code=0
└─ https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Murder-Detection
```

└─ Notebook 3 Complete

Files saved to /content/features/ :

- features_all.csv — full feature matrix (3,163 recordings × 35 features + labels)
- features_train.csv — training split (patient-level 80%)
- features_test.csv — test split (patient-level 20%)
- mfcc_distributions.png — MFCC distributions by outcome
- spectral_distributions.png — spectral feature distributions
- mfcc_correlation.png — MFCC inter-coefficient correlations

Feature set (35 features per recording):

- MFCC mean × 13
- MFCC std × 13
- Spectral centroid, bandwidth, rolloff, flatness × 4
- ZCR, RMS mean, RMS std × 3
- └─ patient_id, site, wav_file, Outcome, Murmur, outcome_binary, split

Next: Module 5 Notebook 4 — XGBoost Classifier + CNN Training

```
In [16]: from google.colab import files
files.download('/content/drive/MyDrive/AAI590_Capstone_Group4/notebooks/A
if False else None

# Save current notebook to Drive
from pathlib import Path
import shutil

for f in __import__('os').listdir('/content'):
    if f.endswith('.ipynb') and ('Feature' in f or 'M4' in f or 'EDA' in
        shutil.copy2(f'/content/{f}',
            '/content/drive/MyDrive/AAI590_Capstone_Group4/noteb
        print(f' Saved to Drive: {f}')
        break
```

```
In [17]: from pathlib import Path

DRIVE_NB_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4/notebo

print('— Notebooks on Drive —————')
nbs = list(DRIVE_NB_DIR.glob('*.ipynb'))
if nbs:
    for nb in nbs:
        size = nb.stat().st_size / 1024
        print(f' {nb.name} ({size:.1f} KB)')
else:
    print(' No notebooks found on Drive')

print(f'\nTotal: {len(nbs)} notebook(s)')
```

```
— Notebooks on Drive —————
  AAI_590_M3_02_EDA.ipynb (932.6 KB)
```

Total: 1 notebook(s)

```
In [18]: from google.colab import _message
import shutil, os
from pathlib import Path

# Force save
_message.blocking_request('save_notebook', request='', timeout_sec=30)

DRIVE_NB_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4/notebo
DRIVE_NB_DIR.mkdir(parents=True, exist_ok=True)

# Find and copy
saved = False
for f in os.listdir('/content'):
    if f.endswith('.ipynb'):
        src = f'/content/{f}'
        dest = DRIVE_NB_DIR / 'AAI590_M4_03_FeatureExtraction.ipynb'
        shutil.copy2(src, str(dest))
        size = Path(src).stat().st_size / 1024
        print(f' {f} ({size:.1f} KB) → Drive')
        saved = True
        break
```

```
if not saved:  
    print('[] Use File → Download → Download .ipynb')  
    print('[] Then upload manually to Drive')
```

- [] Use File → Download → Download .ipynb
Then upload manually to Drive